Call Intake with ZipperGen Studio

A small end-to-end check on a local or remote deployment host

The ZipperGen Authors

Test guide: 28 July 2026

Abstract

This document starts with an empty application directory and ends with a running call-intake deployment. The service reads selected Gmail messages, uses one LLM to classify and extract call data, writes the result to Google Sheets, creates a safe Gmail draft, and records its durable execution state. The purpose is practical: follow the steps and check that the complete Studio path works.

Safe test boundary

Use a dedicated Gmail account or a test address. Keep reply mode `draft` during this guide. Use one empty test spreadsheet. Configure only sender addresses that you control. Do not choose automatic `send` until you have inspected several drafts.

1 What the test proves

The final path is:

Figure 1: The external path exercised by the test.

ZipperGen also records where the Mailbox, Gatekeeper, Extractor, and Table participants are in their projected programs. The deployment owns one SQLite store and one immutable source bundle.

2 Does running on a remote server matter?

For ordinary Studio commands, it does not matter. Run Studio on the same machine that will host the deployment. Paths, model settings, connector bindings, credentials, the durable store, and the managed Python environment then belong to that machine.

Three server details still matter:

1. The server needs outbound HTTPS access to Google and to any remote model provider.

2. Google authorization needs a browser callback. On an SSH-only server, Studio asks you to run one authorization command on the computer that has the browser. You paste the private result back into Studio. No SSH tunnel is needed.
3. A persistent Linux deployment needs a working user `systemd` manager. Some servers require an administrator to enable lingering for the account.

If Ollama runs on the same server as Studio, use `http://127.0.0.1:11434/v1`. No model tunnel is needed. A tunnel is needed only when Ollama runs on a different machine.

3 Prepare the application directory

The outer directory is the application project. The framework checkout is a separate nested repository:

```
mkdir -p "$HOME/Projects"
cd "$HOME/Projects"
mkdir zippergen-call-intake
cd zippergen-call-intake
git init
git clone https://github.com/zippergen-io/zippergen.git zippergen
uv python install 3.12
uv sync --project zippergen --python 3.12 --extra google
```

The `google` extra is needed by the Studio process that checks Gmail and Sheets. It is also needed by the one-time authorization command on the user's own computer. Managed deployments detect the same connector requirements and install this extra automatically.

Start Studio from the outer directory with the nested environment:

```
cd "$HOME/Projects/zippergen-call-intake"
uv run --no-sync --project zippergen zippergen
```

The process uses the nested ZipperGen installation but keeps the outer directory as its project root.

4 Initialize the project and import the workflow

At the Studio prompt, enter:

```
project init call-intake-check
workflow import zippergen/examples/call_intake.py:call_intake
```

Studio copies the workflow to `examples/call_intake.py` in the outer project and selects it. The nested framework repository remains separate.

Check what was imported:

```
workflow files
workflow show communications
workflow show agent Mailbox
workflow show agent Extractor
workflow show full
workflow validate
```

Validation should report four participants: `Mailbox`, `Gatekeeper`, `Extractor`, and `Table`. It should also report the logical connector requirements `call-mailbox` and `call-records`.

5 Record the intended behavior

An imported workflow is visible code, but the fresh project does not yet have a canonical specification. Enter:

```
workflow edit spec
```

Replace the initial writing guide with this text:

```
# Call Intake

Watch a Gmail inbox for call announcements and corrections.

## Participants and behavior

- Mailbox polls Gmail and owns message completion.
- Gatekeeper accepts only configured recipients and certified senders.
- Extractor uses an LLM to classify a message as a call, correction, or other.
- Extractor converts calls and corrections into structured JSON.
- Table upserts records in Google Sheets by a stable call_id.
- Mailbox creates a reply containing the resulting JSON.
- Corrections preserve the call_id and update the existing row.
- Processing is durable and continues polling after each message.

## Safety and configuration

- Gmail and Google Sheets are logical connector requirements.
- Accounts, queries, spreadsheet IDs, tabs, and OAuth tokens stay in Studio.
- Use Gmail draft mode by default.
- Never process a sender that is not explicitly certified.
- Never process a message sent to an unexpected intake address.
- Limit automatic sending to at most ten messages per hour.
```

Save and leave the editor. Implement the current specification, then inspect and validate the result:

```
workflow implement
workflow show spec
workflow show protocol
workflow validate
```

Studio summarizes the changed files, assistant checks, and assistant report. On this machine, deployment later uses the implementation record to detect a specification change. If the project is transferred without private Studio state, deployment warns that no record is available and proceeds after technical validation.

6 Configure the LLM

The only LLM participant is `Extractor`. Use one real model so that the extracted Sheet row is meaningful.

6.1 Option A: Ollama on the same server as Studio

```
model provider configure local
model config create call-extractor
model config check call-extractor
model default call-extractor
model assignments check
```

Accept the default endpoint `http://127.0.0.1:11434/v1`. When Studio asks for a model, enter an identifier returned by the server, for example `qwen2.5:7b`. The configuration check confirms the exact identifier. Studio then asks when the model should be released after its last use. Press Enter for `300` seconds. The call-intake workflow can continue polling Gmail without occupying the model memory. Ollama loads the model again automatically when a new email reaches the Extractor action. Enter `never` only when the model should remain loaded.

6.2 Option B: a hosted model

For OpenAI, enter:

```
model provider configure openai
model config create call-extractor
model config check call-extractor
model default call-extractor
model assignments check
```

Studio asks for the API key privately and then asks for the model name. The same sequence works with `mistral` or `anthropic` as provider. Configure only one option for this test.

7 Prepare Google

Use one Google account for Gmail and the test Sheet.

1. Create or select a Google Cloud project.
2. Enable the Gmail API.
3. Enable the Google Sheets API.
4. Configure the OAuth consent screen.
5. If the application is in external testing mode, add the Gmail account as a test user.
6. Create an OAuth client of type *Desktop app*.
7. Download the client JSON file.
8. Create an empty spreadsheet and name one tab `Calls`.

Keep the downloaded JSON outside the application repository. Do not create a server directory, upload the file, or choose a permanent server filename. The file remains on the computer that performs browser authorization.

8 Configure Gmail and Sheets in Studio

The setup is driven by the selected workflow

Studio has no special case for `call_intake`. The workflow declares logical connector requirements with a name, kind, participant, capabilities, and access level. Those declarations make Studio ask for a Gmail resource and a Google Sheets resource. The workflow's `DeploymentSpec` separately declares the later questions about certified senders, intake recipient, reply mode, rate limit, and polling interval. Selecting a different workflow produces questions from that workflow's own declarations.

Return to the Studio prompt and enter:

```
connector setup
```

Studio asks for:

1. Google authorization on your own computer
2. a Gmail search query
3. a spreadsheet URL or ID
4. the tab name

If Studio and the browser run on the same computer, Studio asks for the downloaded JSON path and opens the browser directly. It does not retain the client file.

If Studio runs on a remote server, it prints a command like:

```
uvx --from \
  'zippergen[google] @ git+https://github.com/zippergen-io/zippergen.git@main' \
  zippergen connector authorize google \
  --scopes gmail.modify,spreadsheets
```

Run exactly the displayed command on your own computer, not on the remote server. `uvx` fetches the current helper into a temporary cached environment, so that computer does not need a ZipperGen checkout or permanent installation. The command asks for the downloaded JSON path, opens Google's consent page, and prints the permissions that Google granted. It then prints one long line starting with `zg-google-v1.`

Copy that whole line and return to Studio. Studio says that nothing will appear while you paste. This is expected because the hidden prompt protects the refresh token inside the result. Press Enter after pasting.

Use the exact scopes that the server displays. The private result has a checksum, so Studio rejects an incomplete paste immediately. After accepting it, Studio shows only non-secret confirmation such as granted scopes and a short OAuth client prefix.

After Google authorization, Studio displays:

```
Gmail search query [is:unread in:inbox]:
```

Choose one intake address that delivers mail to the Google account you just authorized. You will use the same address again when preparing the deployment and when sending the test message. For example, if the authorized account is `alice@example.com`, enter:

```
is:unread in:inbox to:alice@example.com
```

Do not copy `alice@example.com` literally. Replace it with your actual test address. A Gmail plus address such as `alice+call-intake@gmail.com` is also suitable when it delivers to the authorized inbox.

Studio then displays:

```
Google spreadsheet URL or ID:
```

Open the empty test spreadsheet created earlier and paste its complete browser URL. Studio extracts the spreadsheet ID. At the final prompt:

```
Sheet tab [Sheet1]:
```

enter `Calls`. This must match the tab name in the spreadsheet.

> **Why no SSH tunnel is needed**
>
> The browser callback returns to the same computer that runs the authorization helper. That computer creates the portable credential and Studio receives it through the hidden paste. Your own computer does not need network access to the remote server. The OAuth client JSON never reaches the server.

Studio requests Gmail and Sheets scopes because this workflow reads and modifies both services. It verifies the scopes that Google actually granted before storing the resulting refresh credential privately. The project files contain no token. If a later workflow change needs another Google permission, connector checks, runs, and deployments stop with a reauthorization command instead of failing inside the live workflow.

Check the complete connector state:

```
connector
connector provider check google
connector assignments check
```

The expected bindings are:

Requirement	Participant	Kind	Configuration contains
`call-mailbox`	Mailbox	Gmail	account and search query
`call-records`	Table	Sheets	spreadsheet ID and tab

9 Prepare the deployment without starting it

Enter:

```
deploy call-intake-check --no-start
```

Choose these safe values when Studio asks:

Question	Test value
External services	`live`
Certified senders	one exact address that you control
Intake recipient	the address used in the Gmail query
Reply mode	`draft`
Maximum replies per hour	`2`
Polling interval	`15` seconds

Studio creates the immutable bundle, managed Python environment, private deployment secrets, profile, log path, and empty durable store. It does not start the service because of `--no-start`.

Inspect readiness:

```
deploy show call-intake-check
deploy doctor call-intake-check
deploy inspect call-intake-check
```

The store should already exist. The run may say that durable execution has not started yet. That is expected.

10 Send one controlled test message

Send an email that satisfies both configured gates:

- **From**: the exact certified sender
- **To**: the exact intake recipient
- **Subject**: `Test call for formal methods projects`

Example body:

```
The Example Research Fund has opened a call for formal-methods projects.
Applications close on 2026-10-15.
Funding is up to EUR 100,000 for 24 months.
More information: https://example.org/formal-methods-call
```

Leave the message unread until the deployment starts.

11 Start and observe the service

In Studio, enter:

```
deploy start call-intake-check
deploy show call-intake-check
deploy logs call-intake-check
```

Repeat `deploy logs call-intake-check` when you want the newest log output. Reading the log does not stop the managed deployment. After troubleshooting, use `deploy logs reset call-intake-check` to archive the visible history and begin with an empty log view. This does not stop the deployment or alter its durable workflow state.

In another Studio terminal opened from the same outer project directory, use:

```
deploy inspect call-intake-check --watch
deploy trace call-intake-check
deploy storage call-intake-check
```

The inspection view refreshes the durable local-program position for each participant once per second. Press Ctrl-C to close this view. The deployment continues running. During normal polling, Mailbox may be waiting in `wait_briefly`. While the model is active, Extractor may be at an LLM action. Table appears at the Sheets effect while a row is written. `deploy storage` shows how the polling trace and durable recovery data are growing. It does not modify the deployment.

12 Check the external result

The first successful message should produce all of the following:

1. The original Gmail message is no longer unread.
2. The `Calls` tab contains a header and one row.
3. The row contains a stable `call_id`.
4. Gmail Drafts contains a reply with the extracted JSON.
5. `deploy show call-intake-check` reports a running service and an existing store.
6. `deploy trace call-intake-check` shows the participant events.

The `Project alignment` row in `deploy show` compares this deployment with the current project. It covers the specification, accepted workflow, models, idle policy, assistant, connectors, and local model endpoint. Changing the project does not change the running service. Redeploy to apply a reported difference. Enter `project` to see the workflow, runs, and deployments together.

If the Sheet row exists but no draft appears, inspect:

```
deploy logs call-intake-check
deploy doctor call-intake-check
```

If nothing happens, first confirm that the message is unread and matches the configured Gmail query. Then confirm the exact sender and recipient values.

13 Stop or remove the deployment

Stop the service after the test:

```
deploy stop call-intake-check
deploy show call-intake-check
deploy storage call-intake-check
deploy storage compact call-intake-check
```

Stopping preserves the profile, immutable bundle, log, and durable store. Trace retention already runs online and does not require this stop. The optional compaction command removes only completed data covered by recovery snapshots, rotates the service log, and keeps three log archives. It requires the deployment to be stopped. Completed human tasks remain as audit records. Studio also compacts the SQLite file and truncates its WAL safely. The deployment can be started again with:

```
deploy start call-intake-check
```

To remove the service registration but keep recoverable deployment data, use:

```
deploy remove call-intake-check
```

Use the purge option only when the test data is no longer needed:

```
deploy remove call-intake-check --purge
```

Studio asks for the deployment name and confirmation before permanent deployment data is removed. The Google spreadsheet, Gmail messages, drafts, and Google Cloud OAuth client are external resources. Purging a ZipperGen deployment does not delete them.

14 Keeping it alive after SSH logout

On Linux, Studio uses a user `systemd` service when it is available. Check it through Studio:

```
deploy show call-intake-check
deploy doctor call-intake-check
```

Read the `Boot` row. Studio's `deploy start` enables the deployment for automatic startup. If the row says `automatic at server boot`, no manual restart is needed after a server reboot. If it says `automatic after user login`, the service is enabled but account lingering is not. Ask the server administrator whether lingering can be enabled for your account. The usual administrator command is:

```
loginctl enable-linger USERNAME
```

This is account-level server policy, so Studio does not change it automatically. It is not needed for a short test while the SSH session remains open. If only the workflow process crashes while the service manager is running, systemd restarts it automatically after ten seconds. Its durable SQLite state is preserved in both cases.

15 Compact command sheet

```
# Run from the outer project directory
uv run --no-sync --project zippergen zippergen

# Studio commands
project init call-intake-check
workflow import zippergen/examples/call_intake.py:call_intake
workflow edit spec
workflow implement
workflow show communications
workflow show full
workflow validate

model provider configure local
model config create call-extractor
model config show call-extractor
model default call-extractor
model assignments check

connector setup
connector assignments check

deploy call-intake-check --no-start
deploy doctor call-intake-check
deploy start call-intake-check
deploy show call-intake-check
deploy inspect call-intake-check
deploy trace call-intake-check
deploy storage call-intake-check
deploy logs call-intake-check
deploy stop call-intake-check
deploy storage compact call-intake-check
```

Success criterion

The end-to-end check is complete when one controlled unread email becomes one Google Sheets row and one Gmail draft, while Studio reports a running deployment with an existing durable store.